ORM *Mapeo Objeto / Relacional*

- **Desajuste por impedancia:** Fricción o incompatibilidad que surge al intentar conectar un modelo de **programación orientado a objetos** (clases, herencia, métodos) con una **base de datos relacional** (tablas, filas, columnas) ➔ *"Dificultades de pasar de un mundo a otro."*

Almacenar un objeto en una BD

- Ventajas de mapear un objeto: Rapidez y abstracción, olvidándose de cómo guardarlo.
- Desventaja: Tiempo de aprendizaje.

Patrones de diseño

Formas de implementar ORM:

- Patrón Repository: utiliza un repositorio para separar la lógica que recupera los datos y los mapea al modelo de entidades, de la lógica del negocio que actúa en el modelo. El repositorio media entre la capa de fuente de datos y la capa de negocios de la aplicación.
- Patrón Active Record: patrón en el cual, el objeto contiene los datos que representan a una fila (o tupla) de nuestra tabla o vista, además de encapsular la lógica necesaria para acceder a la base de datos. De esta forma el acceso a datos se presenta de manera uniforme a través de la aplicación (lógica de negocio + acceso a datos en una misma clase).

Hibernate

Framework de persistencia de los datos. Cómo llevar los datos objetos a base de datos relacional.
Crea un puente entre nuestra aplicación y la BD.
2 maneras de implementarlo:

1) **Archivos XML:** Se dice los métodos y cómo se resuelve a nivel BD, configurando todos los xml.
2) **Annotations:** Dentro de cada clase se escriben directamente las anotaciones.

NO-SQL Not Only SQL

Sistema de almacenamiento de información **no relacional** que no utiliza esquemas tabulares rígidos de filas y columnas.

BD Relacional: Garantiza **ACID**

- Atomicidad: La transacción es completa o no es. Transacciones a todo o nada.
- Consistencia: El estado de las transacciones está garantizada.
- Independencia: Las transacciones son independientes.
- Durabilidad: Los datos commiteados nunca se pierden.

Ejemplo: MySQL, PostgreSQL, etc…

Big Data

En los últimos años surgió la necesidad de nuevos paradigmas de almacenamiento al aumentar el **Volumen**, **Velocidad** y **Variedad** de los **datos**. ➔ *Crece el volumen de Datos No Estructurados*

BD No Relacional *NO-SQL*

- **Teorema de CAP:** Cumple con 2 de 3 de las siguientes características de este teorema.
  1. Consistencia: Todos los nodos ven los mismos datos al mismo tiempo.
  2. Disponibilidad(Availability): garantiza que cada petición recibe una respuesta acerca de si tuvo éxito o no.
  3. Tolerancia a la partición (partition): el sistema continua funcionando a pesar de la perdida de mensajes
- Suelen **rescindir ACID** para privilegiar Disponibilidad y/o Tolerancia a las Particiones de Red.
- **Escalabilidad:** Capacidad de cambiar su tamaño o configuración para adaptarse a las circunstancias cambiantes en forma ilimitado → Tener + procesamiento y + espacio de almacenamiento

- ➔ Escalabilidad vertical: Al añadir más recursos a un nodo particular del sistema, este mejora en conjunto. (Ej: añadir memoria o un disco duro más rápido a una computadora puede mejorar el rendimiento del sistema global.)
- ➔ Escalabilidad horizontal (NoSQL): Es si al agregar más nodos al mismo, el rendimiento de éste mejora. (Ej: al añadir una computadora nueva a un sistema que balancee la carga entre la antigua y la nueva puede mejorar el rendimiento de todo el sistema.)

- Todas las BD No Relacionales son **muy distintas unas de otras** (no como en las BD relacionales) Tienen *distintas topologías*.
- Es una base **orientada a Documentos**
- **No permite JOINS**
- **Es indexable:** nos va a permitir utilizar índices Geoespaciales como para hacer búsquedas por cercanía por ejemplo.

Tipos de BD No-SQL

BD Clave-Valor	BD de Documentos	BD de Columnas Extendidas	BD de Grafos
Estructura de datos sencilla. Formada por tablas que almacena filas o elementos. No tienen por qué tener los mismos atributos ni la misma cantidad. ➔ *Basados en JSON.*	Almacena elementos en forma de documentos, codificados en *formato JSON*. Cada elemento es un conjunto de pares clave valor y cada documento puede contener otros documentos.	BD distribuida, persistente y ordenada en un mapa multidimensional. Una celda de esta BD viene definida por: - *Rowkey:* Una clave primaria de fila - *Columnkey:* Una clave de columna - *Timestamp:* una marca de tiempo.	BD basada en grafos: conjunto de nodos y relaciones.
Ventajas: - Dinámico y flexible - Estructura genérica Desventajas: - No asegura la integridad del tipo de dato.	Ventajas: - Permite estructuras anidadas - Soporta arreglos y entiende tipos de datos Desventajas: - No asegura la integridad del tipo de dato.	Ventajas: - Flexible para agregar columnas de diferentes filas Desventajas: - No asegura la integridad del tipo de dato.	Ventajas: - La estructura interna es eficiente para almacenar y buscar en estructuras de grafos.
	mongoDB		**Neo4J**

- **Persistencia Políglota:** Utilizar la mejor tecnología de BD para resolver el problema en que esa tecnología es la más eficiente.

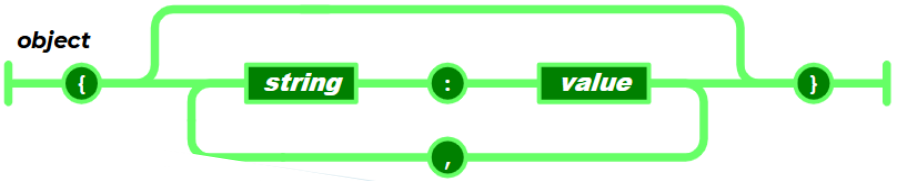
Modelado de BD

➔ **Relacional:** Organiza los datos en **filas**, que no pueden estar anidadas dentro de otras. Las operaciones se basan en almacenar y recuperar filas.
➔ **No Relacional:** Organiza los datos en **AGREGADOS**: Son lo que comparamos como fila en una colección o un registro nuevo. Las estructuras de datos son más complejas.

JSON *MongoDB*

"Registro" o documento estructurado de la siguiente manera:

- Inicia y finaliza con {}
- Clave: String
- Valor: value



➔ **Array:** Separado por corchetes y valores separados por coma → [value, value, value]

Ejemplo:

```
{ "nombre": "Juan Perez",
  "alumno": true,
  "carrera": "Licenciatura en Informatica",
  "materias":
  [
       { "materia": "Laboratorio 2",
         "nota": 7,
         "fecha": "2106-09-20"
       },
       { "materia": "Base de Datos",
         "nota": 9,
         "fecha": "2015-12-20"
       }
  ]
}
```

```
{
   _id: ObjectId("5099803df3f4948bd2f98391"),
   name: { first: "Alan", last: "Turing" },
   birth: new Date('Jun 23, 1912'),
   death: new Date('Jun 07, 1954'),
   contribs: [ "Turing machine", "Turing test", "Turingery" ],
   views : NumberLong(1250000)
}
```

Cada documento incluye un **ID** para identificarlo univocamente.

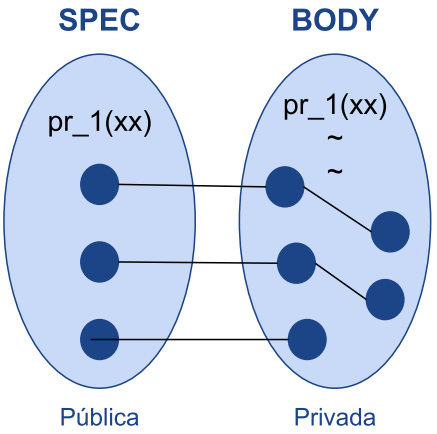
Relaciones

uno-a-uno	uno-a-muchos
	

SQL	MONGODB
CREATE TABLE users (name VARCHAR(128), age NUMBER)	db.createCollection("users")
INSERT INTO users VALUES ('Bob', 32)	db.users.insert({name: "Bob", age: 32})
SELECT * FROM users	db.users.find()
SELECT name, age FROM users	db.users.find({}, {name: 1, age: 1, _id:0})
SELECT name, age FROM users WHERE age = 33	db.users.find({age: 33}, {name: 1, age: 1, _id:0})
SELECT * FROM users WHERE age > 33	db.users.find({age: {$gt: 33}})
SELECT * FROM users WHERE age <= 33	db.users.find({age: {$lte: 33}})
SELECT * FROM users WHERE age > 33 AND age < 40	db.users.find({age: {$gt: 33, $lt: 40}})
SELECT * FROM users WHERE age = 32 AND name = 'Bob'	db.users.find({age: 32, name: "Bob"})
SELECT * FROM users WHERE age = 33 OR name = 'Bob'	db.users.find({$or:[{age:33}, {name: "Bob"}]})
SELECT * FROM users WHERE age = 33 ORDER BY name ASC	db.users.find({age: 33}).sort({name: 1})
SELECT * FROM users ORDER BY name DESC	db.users.find().sort({name: -1})
SELECT * FROM users WHERE name LIKE '%Joe%'	db.users.find({name: /Joe/})
SELECT * FROM users WHERE name LIKE 'Joe%'	db.users.find({name: /^Joe/})
SELECT * FROM users LIMIT 1	db.users.findOne()
SELECT DISTINCT name FROM users	db.users.distinct("name")
SELECT COUNT(*) FROM users	db.users.count()
SELECT COUNT(*) FROM users WHERE AGE > 30	db.users.find({age: {$gt: 30}}).count()
SELECT COUNT(AGE) FROM users	db.users.find({age: {$exists: true}}).count()
UPDATE users SET age = 33 WHERE name = 'Bob'	db.users.update({name: "Bob"}, {$set: {age: 33}}, {multi: true})
UPDATE users SET age = age + 2 WHERE name = 'Bob'	db.users.update({name: "Bob"}, {$inc: {age: 2}}, {multi: true})
DELETE FROM users WHERE name = 'Bob'	db.users.remove({name: "Bob"})

CRUD *(Create-Read-Update-Delete)*	
Create Operation	**Update** Operation
• db.collection.**insert**() • db.collection.**insertOne**() • db.collection.**insertMany**()	• db.collection.**update**() • db.collection.**updateOne**() • db.collection.**updateMany**() • db.collection.**replaceOne**()
Read Operation	**Delete** Operation
• db.collection.**find**()	• db.collection.**remove**() • db.collection.**deleteOne**() • db.collection.**deleteMany**()

Paquetes *Funciones y Procedimientos*



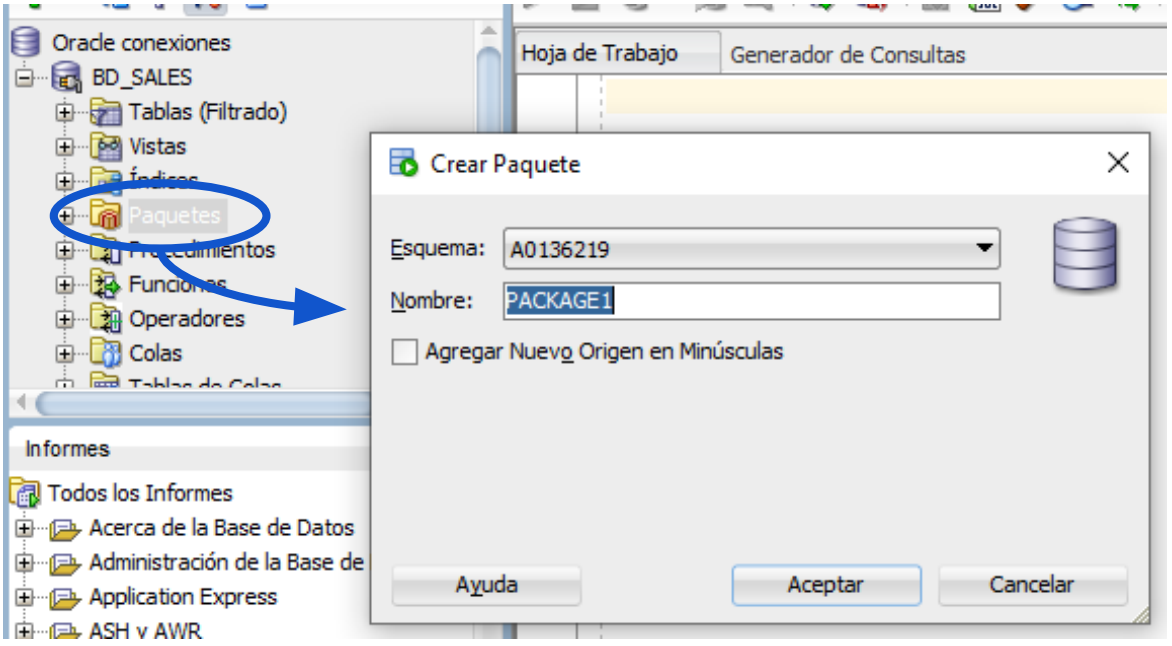
Tiene dos partes:

1) **SPEC:** Especificación de nuestros procedimientos y funciones. ➔ Frima.
   - Es la parte pública.
   - Puedo crear variables globales.
   - Los procedimientos que se llamen igual tienen que tener distinto tipo o número de parámetros
2) **BODY:** Cuerpo con todo el código. ➔ Firma + lo que hace.
   - Es la parte privada.
   - Puedo declarar variables que sean privados.

Por cada firma que tenga en el SPEC, tengo que tener desarrollado en el BODY.

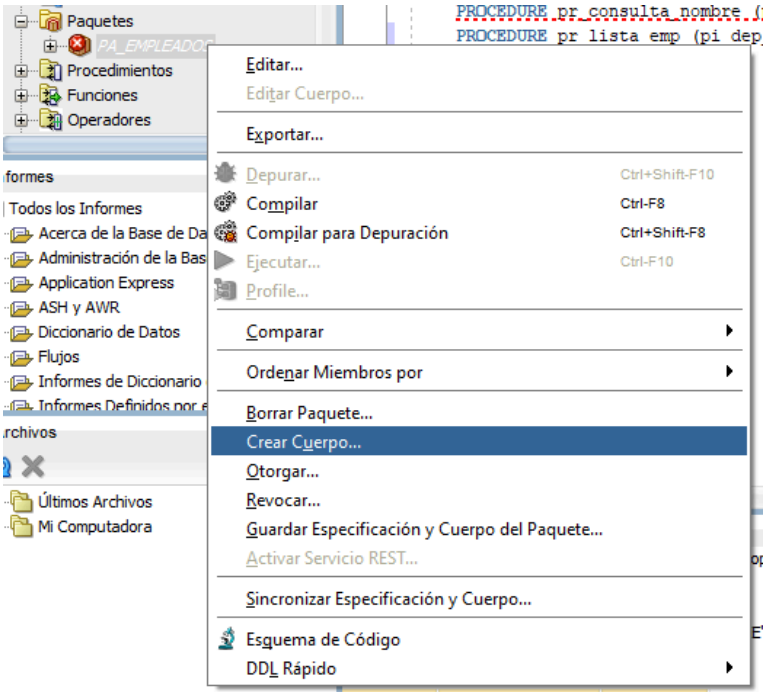
Se puede tener un SPEC y no un BODY, pero nunca solo un BODY.

Crear un Paquete en SQLDeveloper



Crear el BODY (Cuerpo) automáticamente

Haciendo click izquierdo



Ejercicio de ejemplo: Crear un paquete clientes que dé de alta un cliente, con una función privada que valida si existe un cliente o no por id (sino que cancele), y con un procedimiento lista ventas que le pasó un cliente y me enliste las ventas de ese cliente.

SPEC

```
CREATE OR REPLACE
PACKAGE PA_EMPLEADOS AS

PROCEDURE pr_consulta_nombre (pi_emp_id employee.employee%type);
PROCEDURE pr_lista_emp (pi_dep_id department.department_id%type);


END PA_EMPLEADOS;
```

BODY

```
CREATE OR REPLACE
PACKAGE BODY PA_EMPLEADOS AS

  PROCEDURE pr_consulta_nombre (pi_emp_id employee.employee%type) AS
  v_emp_nombre employee.fist_name%type;
  v_emp_apellido employee.last_name%type;
  BEGIN
    SELECT first_name, last_name
    INTO v_emp_nombre, v_emp_apellido
    FROM employee
    WHERE employee_id = pi_emp_id;

    dbms_output.put_line(v_emp_nombre||', '||v_emp_apellido);

  EXCEPTION
    WHEN no_data_found THEN
       dbms_output.put_line('No existe el empleado.');
```

```
    WHEN others THEN
        dbms_output.put_line('Error inesperado al buscar empleado '||SQLERRM);
  END pr_consulta_nombre;

  PROCEDURE pr_lista_emp (po_dep_id department.department_id%type) AS
  BEGIN
    -- TAREA: Se necesita implantación para PROCEDURE PA_EMPLEADOS.pr_lista_emp
    NULL;
  END pr_lista_emp;

END PA_EMPLEADOS;
```

BLOQUE

```
SET SERVEROUTPUT ON
BEGIN
   pa_empleados.pr_consulta_nombre(7869);
END;
```

El **dbms_output.put_line('Parámetro');** es un paquete.procedimiento(parámetro);

Repasar Creación y manejo de Tablas de Memoria.

Temas para el 2° parcial

1. Hibernate (teórico)
2. Las cuatro Topologías de BD
3. BigData